

dl-lab-7

April 18, 2025

1 Labs on Feed Forward Network and Convolutional Neural Network

Dataset- *Dog vs Cat classification.* Take atleast 2000 images of each classes.

Network Architecture: Experiment with FNN and CNN. (You have to find out your optimal architecture)

Training: Train model from scratch. (No transfer learning) Also try the performance of the CNN on the MNIST dataset. (It can be downloaded directly from Keras/Pytorch/TensorFlow, e.g- (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data())

Compare the performance of optimal FNN and CNN on the same dataset.

1.1 Part 1: Dataset Preparation – Dog vs Cat

```
[ ]: from PIL import Image
from torchvision import datasets, transforms

# 1. Create the raw ImageFolder (no split yet)
raw_ds = datasets.ImageFolder(
    "/kaggle/input/microsoft-catsvsdogs-dataset/PetImages",
    transform=transforms.Compose([
        transforms.Resize((128,128)),
        transforms.ToTensor(),
        transforms.Normalize([0.5]*3, [0.5]*3)
    ])
)

# 2. Filter out any samples whose file fails PIL.verify()
valid_samples = []
for path, label in raw_ds.samples:
    try:
        # verify() doesn't load entire image into memory
        Image.open(path).verify()
        valid_samples.append((path, label))
    except Exception:
        pass # skip corrupt file
```

```

# 3. Overwrite the samples & targets on your dataset
raw_ds.samples = valid_samples
raw_ds.targets = [label for _, label in valid_samples]

# 4. Now split & DataLoader as before
from torch.utils.data import random_split, DataLoader

train_n = int(0.8 * len(raw_ds))
test_n = len(raw_ds) - train_n
train_ds, test_ds = random_split(raw_ds, [train_n, test_n])

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True,
    ↪num_workers=2, pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False,
    ↪num_workers=2, pin_memory=True)

```

```

/usr/local/lib/python3.11/dist-packages/PIL/TiffImagePlugin.py:949: UserWarning:
Truncated File Read
    warnings.warn(str(msg))

```

```

[ ]: import torch
from torchvision.datasets import ImageFolder

class SafeImageFolder(ImageFolder):
    def __getitem__(self, idx):
        for _ in range(3):                # try up to 3 times
            path, target = self.samples[idx]
            try:
                img = self.loader(path)
                if self.transform: img = self.transform(img)
                if self.target_transform: target = self.target_transform(target)
                return img, target
            except Exception:
                idx = (idx + 1) % len(self.samples) # move to next sample
        # if still bad after retries, raise
        raise RuntimeError(f"Cannot read image at index {idx}")

# Then use it exactly like ImageFolder:
safe_ds = SafeImageFolder(
    "/kaggle/input/microsoft-catsvsdogs-dataset/PetImages",
    transform=transform
)
# ... split & loader same as before

```

1.2 FNN module

```
[ ]: import torch
import torch.nn as nn
import torch.nn.functional as F

class FNN(nn.Module):
    def __init__(self):
        super(FNN, self).__init__()
        self.fc1 = nn.Linear(3 * 128 * 128, 512)
        self.fc2 = nn.Linear(512, 128)
        self.fc3 = nn.Linear(128, 2) # 2 classes: cat, dog

    def forward(self, x):
        x = x.view(x.size(0), -1) # Flatten
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

1.3 CNN module

```
[ ]: class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 32 * 32, 128)
        self.fc2 = nn.Linear(128, 2)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 64x64
        x = self.pool(F.relu(self.conv2(x))) # 32x32
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

1.4 Training Loop function

```
[ ]: def train_model(model, loader, epochs=5, lr=0.001):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(epochs):
        total_loss = 0
```

```

        correct = 0
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            correct += (outputs.argmax(1) == labels).sum().item()

    acc = 100 * correct / len(loader.dataset)
    print(f"Epoch {epoch+1}/{epochs}, Loss: {total_loss:.4f}, Accuracy: ␣
↪{acc:.2f}%")

```

1.5 Evaluation Function

```

[ ]: def evaluate(model, loader):
    model.eval()
    correct = 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(next(model.parameters()).device), labels.
↪to(next(model.parameters()).device)
            outputs = model(images)
            correct += (outputs.argmax(1) == labels).sum().item()
    acc = 100 * correct / len(loader.dataset)
    print(f"Test Accuracy: {acc:.2f}%")

```

1.6 Run For Cat-Dog

```

[ ]: # Train and evaluate FNN
fnn_model = FNN()
train_model(fnn_model, train_loader, epochs=5)
evaluate(fnn_model, test_loader)

# Train and evaluate CNN
cnn_model = CNN()
train_model(cnn_model, train_loader, epochs=5)
evaluate(cnn_model, test_loader)

```

```

Epoch 1/5, Loss: 445.9852, Accuracy: 59.02%
Epoch 2/5, Loss: 396.2494, Accuracy: 63.63%
Epoch 3/5, Loss: 382.5374, Accuracy: 66.04%
Epoch 4/5, Loss: 374.6168, Accuracy: 67.38%
Epoch 5/5, Loss: 363.2179, Accuracy: 68.85%

```

Test Accuracy: 63.10%
Epoch 1/5, Loss: 358.4291, Accuracy: 69.63%
Epoch 2/5, Loss: 271.8902, Accuracy: 79.67%
Epoch 3/5, Loss: 202.6763, Accuracy: 85.73%
Epoch 4/5, Loss: 120.6151, Accuracy: 92.17%
Epoch 5/5, Loss: 43.6730, Accuracy: 97.64%
Test Accuracy: 77.62%

1.7 PART 2: Try CNN & FNN on MNIST

```
[ ]: from torchvision.datasets import MNIST

transform_mnist = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5])
])

train_mnist = MNIST(root='.', train=True, transform=transform_mnist,
    ↪download=True)
test_mnist = MNIST(root='.', train=False, transform=transform_mnist)

mnist_train_loader = DataLoader(train_mnist, batch_size=64, shuffle=True)
mnist_test_loader = DataLoader(test_mnist, batch_size=64, shuffle=False)
```

Downloading <http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz>
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz>
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz> to ./MNIST/raw/train-images-idx3-ubyte.gz
100%| | 9.91M/9.91M [00:01<00:00, 6.09MB/s]
Extracting ./MNIST/raw/train-images-idx3-ubyte.gz to ./MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz>
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz>
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz> to ./MNIST/raw/train-labels-idx1-ubyte.gz
100%| | 28.9k/28.9k [00:00<00:00, 161kB/s]
Extracting ./MNIST/raw/train-labels-idx1-ubyte.gz to ./MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz>
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz>
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz> to ./MNIST/raw/t10k-images-idx3-ubyte.gz
100%| | 1.65M/1.65M [00:01<00:00, 1.52MB/s]

Extracting ./MNIST/raw/t10k-images-idx3-ubyte.gz to ./MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz>
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz>
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz> to ./MNIST/raw/t10k-labels-idx1-ubyte.gz
100%| | 4.54k/4.54k [00:00<00:00, 5.07MB/s]

Extracting ./MNIST/raw/t10k-labels-idx1-ubyte.gz to ./MNIST/raw

1.8 Update FNN and CNN for MNIST

```
[ ]: # FNN for MNIST (1x28x28)
class FNN_MNIST(nn.Module):
    def __init__(self):
        super(FNN_MNIST, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

# CNN for MNIST
class CNN_MNIST(nn.Module):
    def __init__(self):
        super(CNN_MNIST, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
```

```

self.fc1 = nn.Linear(32 * 7 * 7, 128)
self.fc2 = nn.Linear(128, 10)

def forward(self, x):
    x = self.pool(F.relu(self.conv1(x))) # 14x14
    x = self.pool(F.relu(self.conv2(x))) # 7x7
    x = x.view(x.size(0), -1)
    x = F.relu(self.fc1(x))
    return self.fc2(x)

```

1.9 Train and Evaluate on MNIST

```

[ ]: fnn_mnist = FNN_MNIST()
train_model(fnn_mnist, mnist_train_loader, epochs=5)
evaluate(fnn_mnist, mnist_test_loader)

cnn_mnist = CNN_MNIST()
train_model(cnn_mnist, mnist_train_loader, epochs=5)
evaluate(cnn_mnist, mnist_test_loader)

```

```

Epoch 1/5, Loss: 354.5135, Accuracy: 88.85%
Epoch 2/5, Loss: 186.8385, Accuracy: 94.06%
Epoch 3/5, Loss: 135.5944, Accuracy: 95.73%
Epoch 4/5, Loss: 111.3888, Accuracy: 96.42%
Epoch 5/5, Loss: 93.8164, Accuracy: 96.96%
Test Accuracy: 97.05%
Epoch 1/5, Loss: 192.3677, Accuracy: 93.80%
Epoch 2/5, Loss: 52.4168, Accuracy: 98.29%
Epoch 3/5, Loss: 35.4908, Accuracy: 98.82%
Epoch 4/5, Loss: 28.0366, Accuracy: 99.07%
Epoch 5/5, Loss: 22.9964, Accuracy: 99.20%
Test Accuracy: 98.96%

```

1.10 CONCLUSION

1.10.1 Comparison Table

Model	Dataset	Test Accuracy
FNN	Dog vs Cat	63.10%
CNN	Dog vs Cat	77.62%
FNN	MNIST	97.05%
CNN	MNIST	98.96%